Automatic Differentiation

What and Why

AD differentiates *the program itself* by attaching derivative information to every elementary operation $(+, -, \times, /, \exp, \sin, \ldots)$ and propagating it via the chain rule.

	Finite diff.	**Symbolic diff.**	**AD**
Accuracy	$\mathcal{O}(\varepsilon^{1/2})$–$\mathcal{O}(\varepsilon^{2/3})$ floor	exact (in theory)	$\approx$ function-eval precision
Handles general code	yes	poorly (control flow, swell)	yes
Cost for n-input gradient	$n+1$ f-evals	depends	$\sim$ 2–5$\times$ one f-eval (reverse)

Forward Mode (Tangent Propagation)

Idea: attach a *dual number* $(v,\ \dot{v})$ to each variable, where $\dot{v} = \partial v / \partial x_{\text{seed}}$.

Recipe: **F1** Seed $\dot{x}_{\text{seed}} = 1$, all others $= 0$. **F2** Propagate forward with local rules:

$$(a, \dot{a}) + (b, \dot{b}) = (a+b,\ \dot{a}+\dot{b}), \quad (a, \dot{a}) \cdot (b, \dot{b}) = (ab,\ \dot{a}b+a\dot{b}), \quad g(a, \dot{a}) = \big(g(a),\ g'(a)\dot{a}\big).$$

F3 Read $\dot{f}$ at the output = desired derivative.

Cost: 1 sweep = 1 directional derivative. Need n sweeps for n inputs $\Rightarrow$ **best when n is small**.

Reverse Mode (Adjoint / Backpropagation)

Idea: define *adjoints* $\bar{u}_i = \partial f / \partial u_i$ and propagate them backward from the output to the inputs.

Recipe: **R1** Forward pass: evaluate f, store all intermediates (tape). **R2** Seed $\bar{f} = 1$. **R3** Sweep backward; for each node u_j, sum contributions from all downstream nodes u_i it feeds into:

$$\bar{u}_j = \sum_{i:\, u_j \to u_i} \bar{u}_i \cdot \frac{\partial u_i}{\partial u_j}.$$

R4 At the inputs: $\bar{x} = \partial f / \partial x$, etc.

Cost: 1 forward + 1 backward pass = *full gradient*, independent of input dimension $\Rightarrow$ **best when output is scalar and inputs are many**.

Forward vs. Reverse at a Glance

	Forward	**Reverse**
One sweep gives	1 directional deriv. (Jacobian column)	full gradient ∇f (Jacobian row)
Cost / sweep	$\sim 1\times$ fwd eval	$\sim$ 2–5$\times$ fwd eval
Memory	low (no tape)	must store intermediates
Best for	few inputs, many outputs	many inputs, scalar output
Typical use	Jacobian–vector products	optimisation, HMC, neural nets

Numerical Precision

1. AD differentiates the *floating-point program you ran*, not the idealised math.
2. No subtractive cancellation from differencing (main advantage over finite differences).

3. If the function is stable and well-scaled, AD derivatives are highly reliable.
4. If the function is ill-conditioned, the derivative inherits that sensitivity — AD cannot fix a bad algorithm.

Rule of thumb: AD precision $\approx$ function-eval precision. Scale inputs to $\mathcal{O}(1)$; use stable primitives (`log-sum-exp`, `log1p`, `expm1`).

Practical Considerations

Topic	Guideline
Mode choice	Few inputs $\Rightarrow$ forward; scalar function + many params $\Rightarrow$ reverse
Memory	Reverse stores tape; use *checkpointing* for long computations
Custom ops	Define JVPs (forward) or VJPs (reverse) for specialised kernels
Tools	Python: JAX, PyTorch, TensorFlow; C++: CppAD, Adept, Stan Math; Julia: ForwardDiff, Zygote, Enzyme

Key Takeaways

- AD = local derivative rules + chain rule on the computational graph.
- Precision comparable to evaluating f itself (no differencing artefacts), but subject to FP conditioning.
- Forward mode: cheap per sweep, one derivative direction. Reverse mode: full gradient in one backward pass.
- Choose mode by the shape of the Jacobian: tall $\Rightarrow$ forward; wide $\Rightarrow$ reverse.